



M2 IoT

sujet 1

**Sujet d'évaluation – IoT Système de
capteur de circulation routière**

Auteur : Malik HARRIZ

1 Sujet d'évaluation – IoT : Système de capteur de circulation routière

1.1 Contexte

La gestion du trafic routier est un enjeu majeur dans les villes intelligentes.

Les technologies IoT permettent aujourd'hui de collecter des données en temps réel sur la circulation (nombre de véhicules, vitesse, densité), afin d'optimiser les déplacements et réduire les embouteillages.

Dans ce projet, vous devrez concevoir un **système IoT de monitoring du trafic routier** basé sur une **ESP32**, une **base de données**, et des traitements en **Python incluant du machine learning**.

1.2 Objectif

Développer un système complet permettant de :

- Détecter le passage de véhicules
- Mesurer des indicateurs de trafic (comptage, densité, fréquence)
- Stocker les données dans une base
- Exploiter ces données avec Python
- Mettre en place une analyse en machine learning (prédiction ou classification)

1.3 Description du projet

Vous devez créer un prototype de **capteur de trafic routier connecté**, capable de :

1.3.1 Acquisition des données (ESP32)

- Utiliser une **ESP32** comme microcontrôleur
- Intégrer un ou plusieurs capteurs :
 - Capteur infrarouge (IR)
 - Ultrason ou radar (optionnel)
- Détecter le passage des véhicules
- Compter le nombre de véhicules (traffic count)

1.3.2 Transmission des données

- Envoi des données en temps réel via :
 - HTTP / REST API
 - MQTT (recommandé)

1.4 Base de données - MCD

- Mise en place d'une base de données (MySQL, PostgreSQL, MongoDB...)

1.4.1 Entités de la base de données

1.4.2 Capteur

- id_capteur (PK)
- type_capteur (IR, ultrason, radar...)
- localisation (latitude, longitude ou nom de rue)
- date_installation
- statut (actif / inactif)

1.4.3 Mesure

- id_mesure (PK)
- timestamp
- nombre_vehicules
- vitesse_moyenne

1.4.4 Zone

- id_zone (PK)
- nom_zone
- type_zone (route, intersection, autoroute)
- description

1.4.5 Prediction_ML

- id_prediction (PK)
- timestamp_prediction
- trafic_prevu
- categorie_trafic (faible, moyen, élevé)
- modele_utilise

1.5 Base de données - Relations

1.5.1 Capteur — Mesure

- Un capteur génère plusieurs mesures
- Relation :
Capteur (1,1) — (0,n) Mesure

1.5.2 Zone — Capteur

- Une zone peut contenir plusieurs capteurs
- Relation :
Zone (1,1) — (0,n) Capteur

1.5.3 Mesure — Prediction_ML

- Les prédictions sont basées sur des mesures
- Relation :
Mesure (0,n) — (0,n) Prediction_ML (via dataset ou agrégation temporelle)

1.6 Base de données - MCD simplifié (vue graphique textuelle)

```
[ZONE] (1,n) (1,1) [CAPTEUR]
|
| (1,n)
|
[MESURE]
|
| (n,n)
|
[PREDICTION_ML]
```

1.7 Base de données - Explication du modèle

- **Capteur** : représente chaque ESP32 ou dispositif de détection installé
- **Mesure** : correspond aux données collectées en temps réel (traffic count)
- **Zone** : permet de regrouper les capteurs par localisation (utile pour ML)
- **Prediction_ML** : stocke les résultats du modèle (prédiction de trafic)

1.8 Traitement Python

- Script Python permettant :
 - Récupération des données
 - Nettoyage et transformation
 - Analyse statistique (ex : trafic horaire)
- Visualisation (matplotlib, seaborn, etc.)

1.9 Machine Learning

Vous devez implémenter un modèle simple basé sur les données collectées :

- Prédiction de trafic (régression)

- Classification :
 - trafic faible / moyen / élevé
- Détection d'anomalies

Les systèmes avancés utilisent le machine learning pour analyser la densité du trafic et adapter les décisions en temps réel

1.10 Interface

- Dashboard (Streamlit / Flask)
- Visualisation :
 - trafic en temps réel
 - prédictions ML

1.11 Contraintes techniques

- ESP32 obligatoire
- Code Python obligatoire
- Base de données obligatoire
- Utilisation du machine learning
- Projet fonctionnel de bout en bout

1.12 Livrables

1.12.1 Code

- Code ESP32 (Arduino / C++)
- Scripts Python
- Script base de données

1.12.2 Présentation orale (10 min)

- Problématique
- Architecture
- Description des capteurs
- Structure des données
- Modèle ML utilisé
- Résultats

1.12.3 Démo

- Prototype fonctionnel

1.13 Critères d'évaluation

Critère	Pondération
Fonctionnement capteur ESP32	20%
Transmission des données	15%
Base de données	15%
Traitement Python	15%
Machine Learning	20%
Qualité globale (démonstration)	15%

1.14 Planning conseillé (5 jours)

Jour	Objectif
Jour 1	Conception + capteur ESP32
Jour 2	Transmission + base de données
Jour 3	Traitement Python
Jour 4	Machine learning
Jour 5	Finalisation + présentation

1.15 Bonus (facultatif)

- Détection multi-voies
- Mesure de vitesse
- Intégration caméra / OpenCV

1.16 Résultat attendu

Un système IoT capable de :

- mesurer la circulation routière
- stocker les données
- analyser les flux
- fournir une intelligence décisionnelle